

WEEK 3 DAY (3&4)

Pandas in python

Pandas is a Python library used for data manipulation and analysis. It provides powerful data structures like Series and DataFrame for handling structured data.

Operations in pandas

1. Creating a Series :A Series is a one-dimensional labeled array.

```
import pandas as pd

s = pd.Series([10, 20, 30, 40])
print(s)
```

0	10
1	20
2	30
3	40

dtype: int64

2. Creating a DataFrame: A DataFrame is a two-dimensional table with rows and columns.

```
import pandas as pd

data = {
    "Name": ["A", "B", "C"],
    "Age": [20, 21, 22]
}

df = pd.DataFrame(data)
print(df)
```

	Name	Age
0	A	20
1	B	21
2	C	22

3. Viewing Data: Use functions to display the first or last few rows.

```
print(df.head())  
print(df.tail())
```

	Name	Age
0	A	20
1	B	21
2	C	22
3	D	45
4	E	67

	Name	Age
1	B	21
2	C	22
3	D	45
4	E	67
5	F	89

4. Checking Shape: Returns the number of rows and columns.

```
print(df.shape)
```

```
(6, 2)
```

5. Checking Data Types: Shows the data type of each column.

```
print(df.dtypes)
```

```
Name    object  
Age      int64  
dtype: object
```

6. Selecting a Column: Access a single column using its name.

```
print(df["Name"])
```

```
0    A  
1    B  
2    C  
3    D  
4    E  
5    F  
Name: Name, dtype: object
```

7. Selecting Multiple Columns: Retrieve multiple columns together.

```
print(df[["Name", "Age"]])
```

	Name	Age
0	A	20
1	B	21
2	C	22
3	D	45

8. Selecting Rows using `loc`: `loc` selects rows using labels.

```
print(df.loc[1])
```

Name	B
Age	21

Name: 1, dtype: object

9. Selecting Rows using `iloc`: `iloc` selects rows using integer positions.

```
print(df.iloc[0])
```

Name	A
Age	20

Name: 0, dtype: object

10. Descriptive Statistics: Displays summary statistics of numerical columns.

```
print(df.describe())
```

	Age
count	4.000000
mean	27.000000
std	12.027746
min	20.000000
25%	20.750000
50%	21.500000
75%	27.750000
max	45.000000

11. Checking Missing Values: Identifies missing (null) values in the dataset.

```
print(df.isnull())
```

	Name	Age
0	False	False
1	False	False
2	False	False
3	False	False

```
df.sort_values(by="Age", inplace=True)  
print(df)
```

	Name	Age
0	A	20
1	B	21
2	C	22
3	D	45

12. Filling Missing Values: Replaces missing values with a specified value.

```
df.fillna(0, inplace=True)  
print(df)
```

	Name	Age
0	A	20
1	B	21
2	C	22
3	D	45

13. Dropping Missing Values: Removes rows containing missing values.

```
df.dropna(inplace=True)  
print(df)
```

	Name	Age
0	A	20
1	B	21
2	C	22
3	D	45

GroupBy with Sum 14. Sorting Data: Sorts the DataFrame based on a column.

15.Filtering Data: Returns rows that satisfy a condition

```
print(df[df["Age"] > 20])
```

	Name	Age
1	B	21
2	C	22
3	D	45

16.GroupBy Function: Groups data based on one or more columns and performs aggregate operations

```
import pandas as pd

data = {
    "Department": ["HR", "IT", "HR", "IT", "Sales"],
    "Salary": [30000, 50000, 35000, 60000, 40000]
}

df = pd.DataFrame(data)

print(df.groupby("Department")["Salary"].mean())
```

Department	
HR	32500.0
IT	55000.0
Sales	40000.0

Name: Salary, dtype: float64

17.GroupBy with Sum: Calculates the sum for each group.

```
print(df.groupby("Department")["Salary"].sum())
```

Department	
HR	65000
IT	110000
Sales	40000

Name: Salary, dtype: int64

18.GroupBy with Count: Counts the number of records in each group.

```
print(df.groupby("Department")["Salary"].count())
```

```
Department
HR          2
IT           2
Sales       1
Name: Salary, dtype: int64
```

19.GroupBy with Multiple Aggregations: Performs multiple aggregate functions on grouped data.

```
print(df.groupby("Department")["Salary"].agg(["sum", "mean", "max", "min"]))
```

Department	sum	mean	max	min
HR	65000	32500.0	35000	30000
IT	110000	55000.0	60000	50000
Sales	40000	40000.0	40000	40000

20.Merge: Combines two DataFrames based on a common column.

```
import pandas as pd

df1 = pd.DataFrame({
    "ID": [1, 2, 3],
    "Name": ["A", "B", "C"]
})

df2 = pd.DataFrame({
    "ID": [1, 2, 3],
    "Marks": [85, 90, 95]
})

result = pd.merge(df1, df2, on="ID")
print(result)
```

	ID	Name	Marks
0	1	A	85
1	2	B	90
2	3	C	95

21.Join: Joins two DataFrames based on their index.

```
import pandas as pd

df1 = pd.DataFrame({"Name": ["A", "B", "C"]},
                    index=[1, 2, 3])

df2 = pd.DataFrame({"Marks": [85, 90, 95]},
                    index=[1, 2, 3])

result = df1.join(df2)
print(result)
```

	Name	Marks
1	A	85
2	B	90
3	C	95

22.Concatenate: Combines DataFrames either row-wise or column-wise.

```
import pandas as pd

df1 = pd.DataFrame({"Name": ["A", "B"]})
df2 = pd.DataFrame({"Name": ["C", "D"]})

result = pd.concat([df1, df2])
print(result)
```

	Name
0	A
1	B
0	C
1	D